

fn make_clamp

Sílvia Casacuberta (@silviacasac)

This proof resides in “**contrib**” because it has not completed the vetting process.

Proves soundness of fn make_clamp in [mod.rs at commit 0db9c6036](#) (outdated¹).

1 Hoare Triple

Precondition

Compiler-verified

- Type TA must have trait ProductOrd.
- Type M must have trait DatasetMetric.

Caller-verified

None

Pseudocode

```
1 def make_clamp(  
2   input_domain: VectorDomain[AtomDomain[TA]], input_metric: M, bounds: tuple[TA, TA]  
3 ): #  
4   input_domain.element_domain.assert_non_null() #  
5  
6   # clone to make it explicit that we are not mutating the input domain  
7   output_row_domain = input_domain.element_domain.clone()  
8   output_row_domain.bounds = Bounds.new_closed(bounds)  
9  
10  def clamper(value: TA) -> TA: #  
11    return value.total_clamp(bounds[0], bounds[1])  
12  
13  return make_row_by_row_fallible( #  
14    input_domain, input_metric, output_row_domain, clamper  
15  )
```

Postcondition

Theorem 1.1. For every setting of the input parameters (input_domain, input_metric, bounds) to make_clamp such that the given preconditions hold, make_clamp raises an error (at compile time or run time) or returns a valid transformation. A valid transformation has the following properties:

1. (Data-independent runtime errors). For every pair of members x and x' in input_domain, $\text{invoke}(x)$ and $\text{invoke}(x')$ either both return the same error or neither return an error.

¹See new changes with `git diff 0db9c6036..e8fa1a4 rust/src/transformations/clamp/mod.rs`

2. (Appropriate output domain). For every member x in `input_domain`, `function(x)` is in `output_domain` or raises a data-independent runtime error.
3. (Stability guarantee). For every pair of members x and x' in `input_domain` and for every pair (d_in, d_out) , where d_in has the associated type for `input_metric` and d_out has the associated type for `output_metric`, if x, x' are d_in -close under `input_metric`, `stability_map(d_in)` does not raise an error, and `stability_map(d_in) = d_out`, then `function(x), function(x')` are d_out -close under `output_metric`.

Lemma 1.2. The invocation of `make_row_by_row_fallible` (line 13) satisfies its preconditions.

Proof. The preconditions of `make_clamp` and pseudocode definition (line 3) ensure that the type preconditions of `make_row_by_row_fallible` are satisfied. The remaining preconditions of `make_row_by_row_fallible` are:

- `row_function` has no side-effects.
- If the input to `row_function` is a member of `input_domain`'s row domain, then the output is a member of `output_row_domain`.

The first precondition is satisfied by the definition of `clammer` (line 10) in the pseudocode.

For the second precondition, assume the input is a member of `input_domain`'s row domain. Therefore, by 4, the input is non-null. In addition, since `Bounds.new_closed` did not raise an exception, then by the definition of `Bounds.new_closed`, the bounds are non-null. Thus, by the definition of `ProductOrd`, the preconditions of `total_clamp` are satisfied, so the output is within the bounds. Therefore, the output is a member of `output_row_domain`. $\square$

We now prove the postcondition of `make_clamp`.

Proof. By 1, the preconditions of `make_row_by_row_fallible` are satisfied. Thus, by the definition of `make_row_by_row_fallible`, the output is a valid transformation. $\square$